

Система организации очередей данных

Зачем нужна система очередей

Часто перед разработчиками встает вопрос об организации очередей данных для своего функционала. Если говорить совсем грубо, то главным преимуществом очереди является регулирование нагрузки на функционал, смягчение прямой связи на пиках. Т.е. мы обрабатываем данные со скоростью с которой можем это сделать, если временно данных становится слишком много, то они накапливаются в очереди и будут обработаны позже. Конечно, это допустимо в системах, где лаг в обновлении данных не нарушает принципа их работы.

Зачем нам нужна какая-то своя система? Когда нам потребовался подобный механизм, мы столкнулись с тем, что готового решения у нас нет. Единственной системой, "смягчающей" обмен между сервисами нашего облака, являются асинхронные вызовы (реализованные на [RabbitMQ](#)). С точки зрения очереди она обладает серьезными недостатками:

1. Наличие какой-либо асинхронной очереди (от 10 000 вызовов) воспринимается как авария, т.е. тут очередь это скорее аварийный инструмент.
2. В реальности нет никакой "очередности", более поздние события могут придти раньше более ранних.
3. Нет возможности контролировать темп обработки данных, к обработке асинхронных вызовов привлекаются все воркеры сервиса.
4. В случае ошибки при обработке данных, они будут потеряны (либо надо разрабатывать свои механизмы по их хранению).
5. Нельзя обработать пачку изменений, т.к. мы реагируем на каждый вызов. Почему не воспользоваться готовым решением? Готовых решений для наших сервисов не было. Но можно задаться вопросом — что взять в качестве базы для реализации своей системы? Есть два варианта:
 - о **Kafka** — специализированная система организации очередей. С одной стороны, идеальный вариант, однако, расходы на внедрение данной системы будут очень значительные и руководство пока не готово их понести.
 - о **PgQ** — родная система для PostgreSQL. Есть следующие факторы, из-за которых ее не используют:
 - ее требуется доустанавливать на существующие сервера, а также каким-то образом поддерживать и контролировать;
 - данные в очередях хранятся только в текстовом виде, это значит, что большинство данных придется конвертировать в текстовый формат при записи и деконвертировать при чтении. Понятно, что такая конвертация не бесплатная, а если конвертировать в JSON, то и объем данных значительно увеличится;
 - все равно придется писать систему разбора данной очереди;
 - очень плохая документация по данной системе + мало рычагов администрирования очередей.

Как это работает?

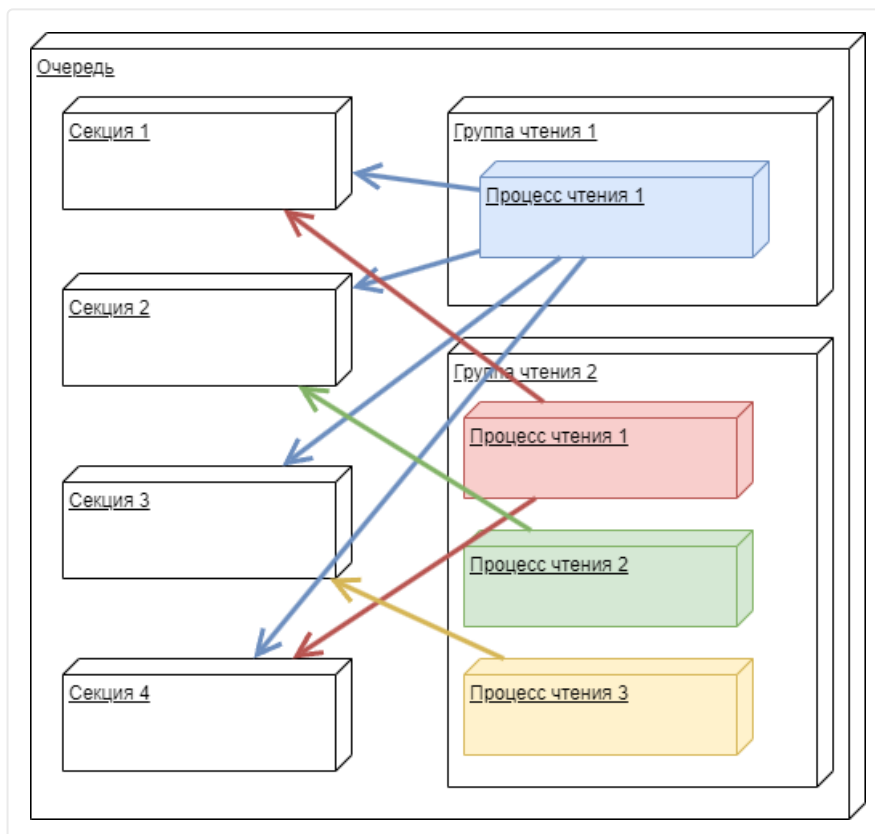
Система очередей - общее название системы буферизации данных. Система предназначена для быстрой вставки данных в буфер с последующим асинхронным чтением данных из буфера и их обработкой.

Очередь - одна функциональная единица системы очередей, состоящая из набора секционированных таблиц, функционала записи данных, функционала чтения данных и функционала по работе с таблицами и данными.

Топик - набор секционированных таблиц в одной очереди

Очередь состоит из секций и групп чтения. Каждая секция очереди — это отдельная таблица, в которой хранятся данные. Данные с одинаковыми идентификаторами всегда попадают в одну и ту же секцию.

Количество секций неизменно, т.е. при проектировании очереди лучше сразу заложить количество секций с запасом. Количество секций определяет максимально возможное количество процессов, которые могут обрабатывать очередь параллельно.



Группа чтения - модель, описывающая процессы чтения данных из топика. Для очереди может быть задано неограниченное количество групп чтения, если нет других ограничений. Группа чтения настраивает и управляет процессом чтения. Она определяет такие параметры, как **количество ридеров, частоту и объём чтения**, а также выбирает **метод обработки данных** и другие настройки, связанные с чтением. Это позволяет гибко адаптировать процесс под конкретные задачи и требования системы.

Группы чтения выполняют независимую обработку очереди, каждая группа читает данные из очереди в полном объеме. Группы чтения состоят из процессов чтения (ридеров).

Ридер (чтец) - один из процессов чтения данных в группе чтения.

Каждый процесс читает только секции, которые определяются на основании количества процессов в группе чтения. Количество процессов чтения в группе может изменяться, но при этом требуется полный перезапуск сервиса, в котором реализована очередь. Таким образом количество процессов чтения можно "повесить" на параметр облака.

Механизм очередей гарантирует следующие положения:

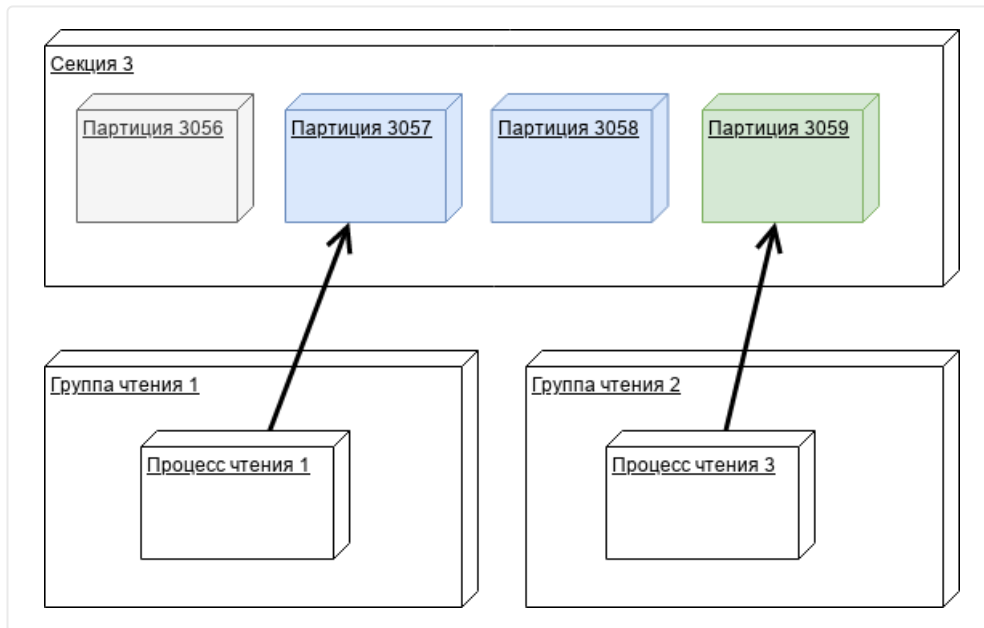
- Если количество процессов чтения не изменяется при перезапуске сервиса, то определенный процесс всегда будет читать из одинаковых секций.
- Данные из секции всегда будут прочитаны только один раз, даже при изменении количества процессов чтения в группе (перебалансировка). Естественно, если процесс чтения завершился без ошибок.
- Можно добавлять новые группы чтения.
- Все манипуляции с группами чтения применяются только при перезапуске сервиса.
- За одну итерацию чтения процесс читает данные (пачку) только из одной секции.
- Если в процессе обработки пачки данных возникла необработанная ошибка, то данные будут прочитаны повторно.
- Если процесс читает данные из нескольких секций, то он это делает последовательно. Если при чтении одной секции возникла ошибка, то процесс чтения перейдет к следующей секции.
- Размер пачки данных определяется в конфигурации группы чтения, у каждой группы могут быть свои настройки.
- Если в группе процессов чтения больше, чем секций в очереди, то часть из них не будут читать данные.

Секции

Секции — это отдельные, динамически партиционируемые таблицы с одинаковой структурой, каждая секция состоит из набора партиций. Секции никак не связаны друг с другом. Другими словами, **секция** - одна таблица в топике.

Партиция - отдельная таблица в секции, партиционирование позволяет разбить одну таблицу на несколько маленьких, что существенно увеличивает скорость чтения и записи в такую таблицу за счет того, что в таких таблицах не деградирует индекс. Такая таблица относительно мала, данные в ней не перезаписываются и после прочтения данных таблица удаляется целиком из файловой системы, что экономит время.

Добавление новых данных всегда выполняется в последнюю партицию, чтение выполняется из всех партиций секции. У каждой очереди есть настройка по времени жизни партиции, если партиция живет дольше этого времени, то при очередной попытке вставки создается (механизмами организации очередей) новая партиция.



Ограничение на использование очередей

- Очереди нельзя использовать в [multitenancy-сервисах](#), т.е. в основном сервисе;
- Нельзя (на постоянной основе) обращаться к таблицам очередей без специального функционала;
- Если вы создаете многосекционные очереди (> 16), то вы должны очень хорошо представлять – зачем вы это делаете?! Большое количество секций замедляет вставку и забивает пул сервиса;
- Не стоит без особой надобности делать очень частое (быстрее 500 мс) чтение!
- В нагруженном состоянии каждый ридер будет занимать один воркер сервиса;
- Крайне нежелательно делать несколько вставок в очередь в одной транзакции.

Виды очередей

Существуют несколько видов очередей, которые отличаются методикой записи, чтения и подтверждения данных. В стандартном редакторе допустимы следующие варианты очереди:

- **Стандартная.** Система очередей контролирует в какую партицию в данный момент записывать данные, из какой партиции читать данные и какие партиции можно удалять. Для стандартной очереди такое управление обеспечивается за счет контроля текущей метки (автоинкрементного первичного ключа) секции.
 - Данные записываются в очередь в порядке поступления
 - Данные считываются из очереди в порядке поступления
 - Подтверждение прочтения выполняется для всей пачки прочитанных данных.
- **Временная.** Для временной очереди контроль метки обеспечивается за счет времени. Каждая новая партиция нумеруется не последовательным числовым индексом, как стандартная, а временем в формате timestamp. Чтение данных из партиции происходит при достижении серверного времени такого времени, какое указано в номере партиции.
 - Данные записываются в очередь в порядке, определяемым специальным полем **queue_time**, куда записывается время. Если время меньше текущего на момент записи, то оно заменяется текущим.
 - При чтении данные читаются также с учетом поля **queue_time** (по возрастанию), читаются только те данные, у которых значение поля **queue_time** меньше текущего времени.
 - Подтверждение прочтения выполняется для всей пачки прочитанных данных.

Стандартное подтверждение чтения данных – механизм контроля смещения метки обеспечивается за счет подтверждения чтения набора данных из секции. Если прикладной метод, обрабатывающий пачку данных вернул положительное подтверждение, то для данного метода метка смещается вперед. Такой механизм гарантирует, что данные будут прочитаны всеми прикладными методами, и только после этого отработанные секции будут удалены.

Метод обработчик - прикладной метод БЛ, который на вход получает, прочитанные из очереди данные и контекст чтения. Метод производит необходимые манипуляции с данными и подтверждает чтение устанавливая нужные поля в контексте чтения. Наименование метода обработчика указывается при создании очереди.

Метка очереди - целочисленное значение, которое присваивается каждой новой записи в очереди. Записи которые должны быть прочитаны позже всегда имеют большую метку чем записи которые должны быть прочитаны раньше.

- Стандартная с удалением

- Данные записываются в очередь в порядке поступления
- Данные считываются из очереди в порядке поступления
- Подтверждение выполняется для массива записей, т.е. из прочитанных записей можно подтвердить нужный вам набор. Неподтвержденные записи будут прочитаны повторно.

- Временная очередь с удалением

- Данные записываются в очередь в порядке, определяемым специальным полем **queue_time** куда записывается время. В отличие от простой временной очереди система не корректирует значение поля **queue_time** при вставке, если оно меньше текущего времени.
- При чтении данные читаются также с учетом поля **queue_time**, при этом читаются только те данные, у которых значение поля меньше текущего времени.
- Подтверждение выполняется для массива записей, т.е. из прочитанных записей можно подтвердить нужный вам набор. Неподтвержденные записи будут прочитаны повторно.

Подтверждение путем удаления - данные в партиции удаляются сразу после успешного их прочтения, этот механизм оптимизирует хранение данных, но позволяет читать их только одним методом.

Стандартная очередь

Для решения большинства задач подойдет стандартная очередь, напомним ее основные свойства:

- Данные записываются в очередь в порядке поступления,
- Данные считываются из очереди в порядке поступления,
- Подтверждение прочтения выполняется для всей пачки прочитанных данных.

Также отметим, что данная очередь самая производительная, по сравнению с другими типами очередей.

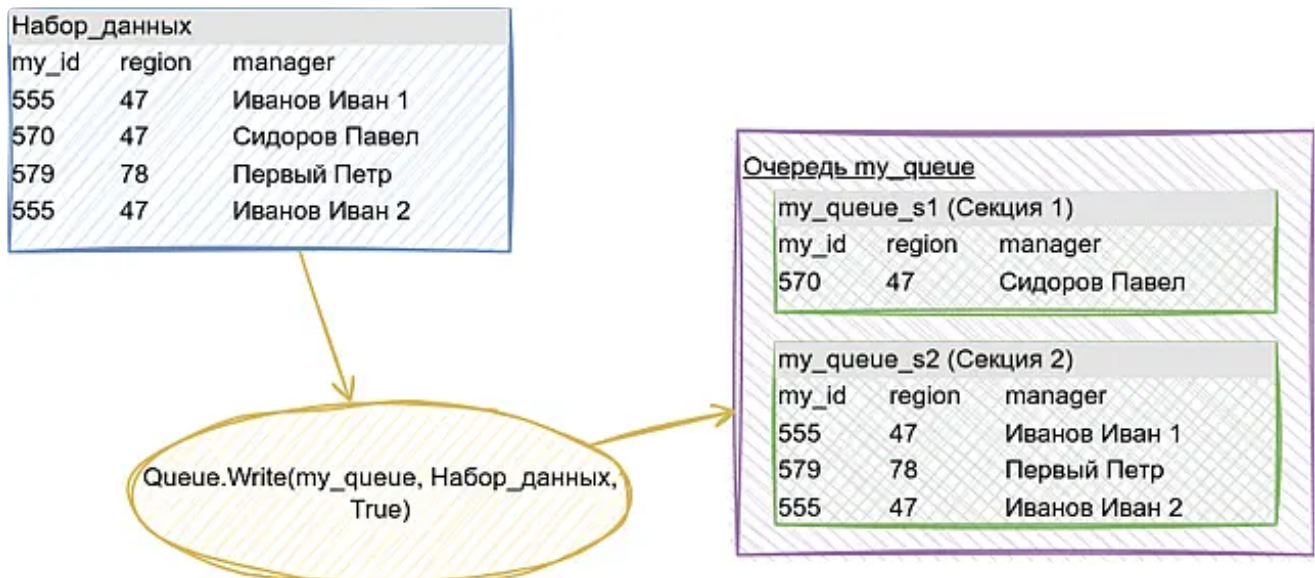
The screenshot shows the Genie web interface. At the top, there's a navigation bar with 'Genie', 'Project', 'Edit', 'Report', 'Settings', and 'shop-tensor SDK: 24.5200'. Below it, a breadcrumb trail reads 'shop-tensor > shop.tensor.ru-srv > Shop > Shop > Shop.queuex'. The main content area is titled 'my_first_queue...' and contains a 'QUEUE TOPICS' section on the left with a search bar and a list of topics, including 'my_first_queue' with the comment 'Моя первая очередь'. On the right, the 'Queue topic' configuration page is shown for 'my_first_queue'. It has a 'Main' section with several attributes: 'Queue type' is set to 'Standard' (highlighted with a red box), 'Name' is 'my_first_queue', 'Comments' is 'Моя первая очередь', 'Responsible' is 'Думп Ю.П.', 'Section count' is '2', 'Id field' is 'id', 'Confirm type' is 'Standard' (highlighted with a red box), and 'Reader type' is 'Standard'.

Для создания данного типа очереди необходимо установить значения атрибутов:

- **Queue Type:** Standard
- **Confirm Type:** Standard

Запись данных в стандартную очередь

При вставке данных в очередь, записи распределяются по секциям на основании поля идентификатора. Если оно не задано, то случайным образом.



- Механизм записи сам распределит входящие данные по секциям, на основании поля-идентификатора. Кстати, данное поле может быть любым элементарным типом.
- Механизм гарантирует, что записи с одинаковым идентификатором попадут в одну секцию.
- Механизм гарантирует, что записи будут обработаны в порядке поступления в очередь.

Для записи используют платформенные методы из модуля **Queue**:

- **Queue.Write** - записывает **RecordSet** данных в очередь. На вход следует передать следующие параметры:
 - **topic** - наименование очереди,
 - **rs** - RecordSet со вставляемыми данными,
 - **native_format** - признак приведения данных к формату очереди. True - данные будут приведены к формату таблицы-очереди (могут не совпадать типы, отсутствовать или присутствовать лишние столбцы), такой вариант устойчив к ошибкам, но довольно медленный, т.к. происходит копирование данных. False - значительно быстрее, но если формат данных не совпадет, то вы получите ошибку при вставке.
- **Queue.WriteRecord** - записывает **Record** в очередь. На вход следует передать следующие параметры:
 - **topic** - наименование очереди,
 - **rec** - **Record** с вставляемыми данными,
 - **native_format** - признак приведения данных к формату очереди. True - данные будут приведены к формату таблицы-очереди (могут не совпадать типы, отсутствовать или присутствовать лишние столбцы).



Вставка в очередь допустима в транзакции, но (**ВАЖНО!!!**) в рамках одной транзакции вставку в **ОДНУ** очередь можно делать только **ОДИН** раз, и желательно, ближе к концу транзакции. Если не выполнять данное правило, то можно словить **deadlock**.

Чтение данных из стандартной очереди

Для чтения данных из очереди требуется разработать метод БЛ со следующими характеристиками:

- На входе принять два параметра (RecordSet - прочитанные данные и Record - контекст чтения),
- Обработать пришедшие данные,
- В контексте проставить поле **commit = True**,
- Вернуть контекст,
- Как правило, метод-обработчик создается внутренним (Internal),
- Прочитанные данные не удаляются сразу из таблицы. Контроль за прочитанными данными выполняется системой очередей.

```
1 # Код метода БЛ из orx (Genie)
2 def Test.ReadInt(data, context_rec):
3     # Пример обработчика из теста очередей
```

```

4         for rec in data:
5             # Проходим по выборке и выполняем необходимые
манипуляции с данными,
6             # в тесте мы их вставляем в проверочную таблицу
7             SqlQuery("""INSERT INTO public.test_topic_int(
8                 mark, part, id, field_string,
field_int, section)
9                 VALUES ($1, $2, $3, $4, $5, $6);""",
10             rec.mark, rec.part, rec.id, rec.field_string, rec.field_int,
context_rec.section)
11             # После обработки данных подтверждаем прочтение
12             context_rec.commit = True
13             # Обязательно возвращаем контекст, если этого не сделать, то
данные будут прочитаны заново
14             return context_rec

```



Если не подтвердить чтение или не вернуть контекст, то на следующей итерации будет прочитан тот же кусок данных. Бывают бизнес-сценарии, когда данные не надо подтверждать на конкретной итерации чтения (например, временная неготовность какой-либо системы к обработке данных).

Частичное подтверждение прочтения

При стандартном подтверждении прочтения, есть возможность подтвердить только часть прочитанных данных.



ВАЖНО!!!

Частично подтвердить можно только "левую" часть данных.

В пришедших данных всегда присутствует поле **mark** - это метка записи в очереди, пришедшие данные упорядочены по данной метке. Т.е. если вы последовательно обрабатываете выборку и в какой-то момент поняли что хватит, то вы просто берете значение **mark** из последней обработанной записи и проставляете его в поле **max_mark** в контексте чтения. Новая итерация начнется с метки которую вы установили, т.е. неподтвержденная часть данных будет прочитана заново.

```

1  # Код метода БЛ из orx (Genie)
2  def Test.ReadInt150(data, context_rec):
3      # Пример из теста очередей, читаем только первые 150 записей
4      i = 0
5      for rec in data:
6          # Последовательно обрабатываем выборку (она упорядочена
по полю mark)
7          if i < 150:
8              SqlQuery("""INSERT INTO public.test_topic_int(mark, part,
id, field_string, field_int)
9                  VALUES ($1, $2, $3, $4,
$5);""",
10              rec.mark, rec.part, rec.id,
rec.field_string, rec.field_int)
11              # Проставляем в max_mark текущую метку из
выборки
12              context_rec.max_mark = rec.mark
13              i += 1
14          else:
15              # Тут мы решили что хватит
16              break
17          # Далее поступаем стандартно
18          context_rec.commit = True

```

Очередь с приоритетом по времени

Предназначена для обработки данных в порядке определяемом временным полем, а также не раньше, чем наступит определенный момент времени. Напомним основные свойства данной очереди:

- Данные записываются в очередь в порядке, определяемым специальным полем **queue_time**, куда записывается время. Если время меньше текущего на момент записи, то оно заменяется текущим.
- При чтении данные читаются также с учетом поля **queue_time** (по возрастанию), при этом читаются только те данные, у которых значение поля **queue_time** меньше текущего времени.
- Подтверждение прочтения выполняется для всей пачки прочитанных данных.

Данный тип очереди полезен для организации отложенной обработки элементов.

The screenshot shows the Genie web interface. The top navigation bar includes 'Genie', 'Project', 'Edit', 'Report', 'Settings', and 'shop-tensor SDK: 24.5200'. The breadcrumb trail is 'shop-tensor > shop.tensor.ru-srv > Shop > Shop > Shop.quex'. The left sidebar shows 'QUEUE TOPICS' with a search bar and a list containing 'my_first_queue' with the subtitle 'Моя первая очередь'. The main panel is titled 'Queue topic' and shows the configuration for 'my_first_queue'. The 'Queue type' is set to 'Time' and the 'Confirm type' is set to 'Standard', both highlighted with red boxes. Other fields include 'Name' (my_first_queue), 'Comments' (Моя первая очередь), 'Responsible' (Думп Ю.П.), 'Section count' (2), 'Id field' (id), and 'Reader type' (Standard).

Для создания данного типа очереди необходимо установить значения атрибутов:

- **Queue Type:** Time
- **Confirm Type:** Standard

При развороте данной таблицы появится еще одно служебное поле **queue_time**.

Запись в очередь с приоритетом по времени

Методы записи аналогичны [стандартной очереди](#). Однако система распределения работает по другому.

Набор_данных			
my_id	region	manager	queue_time
555	47	Иванов Иван 1	12:00
570	47	Сидоров Павел	11:30
579	78	Первый Петр	11:20
555	47	Иванов Иван 2	11:00

Queue.Write(my_queue, Набор_данных, True)

Очередь my_queue			
my_queue_s1			
my_id	region	manager	queue_time
570	47	Сидоров Павел	11:30
my_queue_s2			
my_id	region	manager	queue_time
555	47	Иванов Иван 2	11:00
579	78	Первый Петр	11:20
555	47	Иванов Иван 1	12:00

Записи распределяются по полю **queue_time** независимо от порядка прихода, при этом если время записи меньше текущего времени на момент вставки, то оно заменятся текущим.

Чтение из очереди с приоритетом по времени

Чтение производится аналогично [стандартной очереди](#), без каких-либо дополнительных особенностей.

Особенности очередей с приоритетом по времени

1. Во временной очереди чтение кратковременно блокирует вставку.
2. Во временной очереди категорически запрещается делать несколько вставок в одну очередь в одной транзакции.
3. Если вы делаете много вставок в далекое будущее, то в таблице секции вашей очереди будет создано много партиций, это может снизить производительность некоторых процессов (например, определения размера очереди или получения структуры БД для explain). Для сокращения количества партиций увеличивайте их размер, например, до суток (параметр **Part lifetime**).

Стандартная очередь с удалением

Из названия очереди, вытекает принцип подтверждения записей - они физически удаляются из таблицы в момент подтверждения. Предназначена для реализации возможности выборочного подтверждения прочитанных записей, причем принятие решения о подтверждении конкретных записей принимается в методе-обработчике. Более того, можно по отдельности подтверждать конкретные записи в рамках обработчика, что позволяет выполнять параллельную обработку прочитанных записей.

Все неподтвержденные записи будут прочитаны повторно.

Для создания данного типа очереди необходимо задать значения атрибутов:

Genie Project Edit Report Settings shop-tensor SDK: 24.5200

shop-tensor > shop.tensor.ru-srv > Shop > Shop > Shop.quex

my_first_queue...

QUEUE TOPICS ⓘ

Найти 🔍 +

my_first_queue
Моя первая очередь

Queue topic 📄

^ Main

Queue type ⓘ	Standard
Name ⓘ	my_first_queue
Comments ⓘ	Моя первая очередь
Responsible	Думп Ю.П. ✕
Section count ⓘ	2
Id field ⓘ	id
Confirm type ⓘ	Delete
Reader type ⓘ	Standard

- Queue Type: Standard
- Confirm Type: Delete

Запись в стандартную очередь с удалением

Запись ничем не отличается от записи в [стандартную очередь](#).

Чтение из стандартной очереди с удалением

В методе-обработчике очереди с удалением есть серьезное отличие от [стандартной очереди](#), в контексте к вам придет поле **confirm**, которое является массивом меток которые следует подтвердить после прочтения. По умолчанию в массиве содержатся все метки в пришедшей выборке.

```

1 # Пример кода из теста системы очередей
2 def Test.ReadIntDel(data, context_rec):
3     # В тестовом примере подтверждаем записи с четной меткой
4     escape_ids = []
5     for rec in data:
6         if rec.id % 2:
7             SqlQuery("INSERT INTO public.test_topic_int(mark, part, id,
8 field_string, field_int) VALUES ($1, $2, $3, $4, $5);",
9             rec.mark, rec.part, rec.id, rec.field_string,
10            rec.field_int)
11        # Добавляем четную запись в массив подтвержденных меток
12        escape_ids.append(rec.mark)
13
14 # ОБЯЗАТЕЛЬНО!!! Проставляем массив подтвержденных записей в поле
15 confirm в контексте
16 context_rec.confirm = escape_ids
17 # ОБЯЗАТЕЛЬНО!!! Проставляем поле с подтверждением прочтения
18 context_rec.commit = True
19 return context_rec

```

Особенности стандартной очереди с удалением

1. В стандартной очереди с удалением может быть только одна группа чтения.

2. Очередь с удалением более ресурсоемкая, чем обычная очередь.
3. Можно перейти от обычной очереди к очереди с удалением без каких-либо конвертаций и наоборот. Однако стоит учесть, что при переходе от обычной очереди к очереди с удалением возможно повторное прочтение данных, которые были помечены как прочитанные в очереди без удаления.

Временная очередь с удалением

Сочетает в себе свойства [временной очереди](#) и [очереди с удалением](#). Была разработана, т.к. обычный механизм подтверждения прочтения для временной очереди слишком сложный. Рекомендуется использовать именно такой тип временной очереди, за исключением очередей с несколькими группами чтения.

Основные свойства данного типа очереди:

- Данные записываются в очередь в порядке, определяемым специальным полем **queue_time**, куда записывается время. В отличие от простой временной очереди система не корректирует значение поля **queue_time** при вставке, если оно меньше текущего времени.
- При чтении данные читаются также с учетом поля **queue_time**, при этом читаются только те данные, у которых значение поля **queue_time** меньше текущего времени.



ВАЖНО!!!

В данном типе временной очереди чтение не блокирует запись.

- Подтверждение выполняется для массива записей, т.е. из прочитанных записей можно подтвердить нужный вам набор. Неподтвержденные записи будут прочитаны повторно.

The screenshot shows the Genie web interface. The top navigation bar includes 'Genie', 'Project', 'Edit', 'Report', 'Settings', and 'shop-tensor SDK: 24.5200'. The breadcrumb trail is 'shop-tensor > shop.tensor.ru-srv > Shop > Shop > Shop.quex'. The left sidebar shows a tree view with 'my_first_queue...' expanded, containing 'QUEUE TOPICS' and a search bar. Below the search bar, 'my_first_queue' is listed with the description 'Моя первая очередь'. The main panel is titled 'Queue topic' and shows the configuration for 'my_first_queue'. The 'Main' section contains several fields: 'Queue type' (set to 'Time'), 'Name' (set to 'my_first_queue'), 'Comments' (set to 'Моя первая очередь'), 'Responsible' (set to 'Думп Ю.П. x'), 'Section count' (set to '2'), 'Id field' (set to 'id'), 'Confirm type' (set to 'Delete'), and 'Reader type' (set to 'Standard'). The 'Queue type' and 'Confirm type' fields are highlighted with red boxes.

Для создания данного типа очереди необходимо установить значения атрибутов:

- **Queue Type:** Time
- **Confirm Type:** Delete

Запись во временную очередь с удалением

Запись выполняется аналогично записи в обычную [временную очередь](#), однако есть одна важная особенность - если значение в поле **queue_time** меньше текущего момента времени, то оно не изменяется и вставляется как есть. При этом читаются данные всегда с сортировкой по времени, значит новые записи со старой меткой будут прочитаны раньше чем более старые записи (если у них поле **queue_time** больше).

Чтение из временной очереди с удалением

Чтение аналогично чтению [стандартной очереди с удалением](#).

Дополнительные возможности

Ускорение чтения

Чтение ридером данных из секции очереди выполняется не чаще чем значение **Read Temp** (**read_temp_msecs** если очередь объявлена не в queh) настроенное в группе чтения. Напомню, если у вас стоит темп 1000 мс, а итерация чтения прошла за 200 мс, то 800 мс ридер будет ждать. Если же итерация чтения прошла за 1200 мс, то следующая запустится сразу.

Начиная с версии 25.3000 появилась возможность запустить следующую итерацию чтения, игнорируя установленный таймаут. Это полезно в ситуациях, когда вы понимаете что очередь обрабатывается слишком медленно и ждать установленный таймаут опасно. Для ускорения установите параметр **no_timeout = True** в контексте чтения из очереди.

```
1     def read_queue_method(data, ctxt):
2         # Выполняем код
3         if data.Size() > 1000: #Тут мы решили что надо ускорится
4             ctxt.no_timeout = True
5
6         ctxt.commit = True
7         return ctxt
```

Следующая итерация чтения начнется сразу после предыдущей.



Опция действует только на следующую итерацию



ВАЖНО!!! Если ридер читает из нескольких секций, то на следующую итерацию чтение будет выполнено из этой же секции! В обычной ситуации секция меняется на каждую итерацию

Данная опция позволит сделать процедуру чтения более "ленивой", т.е. можно будет поставить заведомо больший интервал чтения, но при необходимости "ускориться".